Estudiantes:
Axel Javier Rosas Rodríguez | A01738607
Elian Cantalapiedra Sabugal | A01738462
Edwin Emmanuel Salazar Meza | A01738380

Profesor:
Rigoberto Cerino Jiménez

Nombre de la Institución:
Instituto Tecnológico y de Estudios Superiores de Tecnológico de Monterrey

Materia:
Implementación de Robótica Inteligente

Fecha:
5 de Abril de 2026

Título del trabajo:
Reto semanal 1. Manchester Robotics

Resumen

El presente trabajo tiene como objetivo implementar la conexión, comunicación y control de un robot móvil Puzzle Bot utilizando herramientas de ROS2, con el fin de operarlo de manera remota y analizar su comportamiento ante distintos comandos de movimiento. Para ello, se abordaron conceptos fundamentales como la arquitectura del robot, la comunicación mediante SSH y el uso de tópicos en ROS2.

El Puzzlebot integra diversos componentes de hardware y software, como motores con encoders, sensores y un microcontrolador, los cuales interactúan a través de una arquitectura distribuida basada en ROS. Esto permite el intercambio de información en tiempo real mediante nodos y tópicos. Para establecer la comunicación con el robot, se configuró una red inalámbrica tipo hotspot desde el Jetson Nano, asignando una dirección IP que permitió la conexión remota mediante el protocolo SSH.

Para el control del robot, se utilizó la herramienta Teleop Twist Keyboard, que permite enviar comandos de velocidad lineal y angular mediante el teclado. Finalmente, se realizaron pruebas de desempeño para evaluar los límites de velocidad y respuesta del robot, confirmando el correcto funcionamiento del sistema y su capacidad de operación remota.

Objetivos

Objetivo General:

- Implementar la conexión, comunicación y control de un robot móvil Puzzle Bot utilizando herramientas de ROS2, con el fin de operar el robot de manera remota y analizar su comportamiento en diferentes comandos de movimiento.

Objetivos específicos:

- Comprender la arquitectura y funcionamiento del puzzle bot
- Establecer una conexión remota al puzzle bot mediante ssh
- Identificar el funcionamiento de los tópicos de ros2
- Implementar la teleoperación mediante teleop twist keyboard
- Verificar el desempeño del robot al modificar sus parámetros

Introducción

El Puzzle Bot es un robot móvil modular diseñado con una arquitectura que integra diversos elementos de hardware y software. Entre sus componentes principales se encuentran un microcontrolador, motores con encoders, sensores para la percepción del entorno y módulos de comunicación. Estos elementos trabajan de manera interconectada mediante un sistema basado en ROS, lo que permite la gestión de la información a través de nodos que intercambian datos en tiempo real.

Para lograr la interacción con el robot, es necesario establecer una comunicación remota confiable. El protocolo SSH permite acceder de manera segura al sistema del Puzzlebot

desde una computadora externa. SSH funciona mediante una conexión cifrada que posibilita la ejecución de comandos, la transferencia de archivos y la gestión remota del robot, facilitando así el desarrollo, monitoreo y control del sistema sin necesidad de una conexión física directa.

Una vez establecida la comunicación, es posible controlar el robot utilizando herramientas específicas dentro del ecosistema ROS. Utilizamos Teleop Twist Keyboard, la cual permite enviar comandos de velocidad al robot mediante el teclado. Esta herramienta publica mensajes del tipo Twist, que contienen información sobre la velocidad lineal y angular, permitiendo así controlar el movimiento del Puzzle Bot en tiempo real. El robot, a través de sus nodos de control, interpreta estos comandos y los traduce en señales que regulan la velocidad de los motores.

En conjunto, la integración del Puzzlebot, la comunicación mediante SSH y el uso de herramientas como Teleop Twist Keyboard constituyen un sistema completo que permite no solo el control remoto del robot, sino también la comprensión de conceptos clave en robótica, como la arquitectura distribuida, la comunicación entre procesos y el control de movimiento. Este enfoque proporciona una base sólida para el desarrollo de aplicaciones más avanzadas en automatización y robótica móvil.

Metodología

1. Preparación del sistema Jetson Nano

- Descargar la imagen del sistema.
- Flashear la imagen en la tarjeta microSD.
- Insertar la microSD en el Jetson Nano y encender el dispositivo.
- Completar la configuración inicial del sistema (usuario, contraseña, red, etc.).

2. Configuración de la red Wi-Fi

- Configurar los parámetros de red:
    - SSID: I_Robloxianas
    - Password: Puzzlebot72
- Activar el hotspot.

3. Conexión remota vía SSH

- Conectar la computadora externa a la red Wi-Fi.
- Abrir una terminal en la computadora externa y ejecutar:
    - ssh puzzlebot@10.42.0.1

4. Ejecución del agente micro-ROS

```
$ ros2 run micro-ros-agent micro-ros-agent serial -dev /dev/ttyUSB0
```

5. Verificación de comunicación ROS 2

- En la terminal, ejecutar “ros2 topic list” y confirmar los siguientes tópicos:
    - /cmd_vel
    - /VelEncL
    - /VelEncR

6. Publicación y lectura de tópicos

7. Control remoto con teclado (Teleoperación)

- Instalación del paquete

```
$ sudo apt install ros-humble-teleop-twist-keyboard -y
```

- Ejecución del paquete

```
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

8. Pruebas de desempeño del robot

- Evaluar:
    - Velocidad máxima y mínima lineal
    - Velocidad máxima y mínima angular
- Registrar resultados

Solución del problema

Una vez instalado el sistema operativo proporcionado por MCR2 para el Puzzle Bot, comprobamos que todos sus componentes estén en perfecto estado, y se mantenga una comunicación entre ellos.

Imagen 1. Comprobación del funcionamiento de los componentes

Una vez comprobado el funcionamiento de todos los componentes del Puzzle Bot, conectados el adaptador wifi y lo configuramos como un Hotspot:

- SSID: I_Robloxianas
- Password: Puzzlebot72

Posteriormente, desde otra computadora, abrimos una terminal y nos conectamos mediante ssh al Puzzle Bot. Una vez conectados, iniciamos el agente de micro ros y vemos la existencia de los siguientes tópicos:

```
puzzlebot@jetson:~$ ros2 topic list
/LaserDistance
/ServoAngle
/VelocityEncL
/VelocityEncR
/cmd_vel
/parameter_events
/robot_vel
/rosout
```

Imagen 2. Verificación de los tópicos de ROS2

Imagen 3. Inicialización del agente de micro-ROS

Resultados

Para determinar el rango de valores máximos y mínimos, se ejecutó el nodo teleop_twist_keyboard, el cual permite controlar el movimiento del Puzzlebot de forma manual y ajustar las velocidades lineal y angular (como se observa en la imagen 2). Debido a que no contábamos con un rango predeterminado de operación, realizamos un proceso de experimentación mediante prueba y error para identificar los límites en los que el robot respondía de manera eficiente y sin latencia a los comandos de avance, retroceso y giro.

Tras diversas pruebas, se determinó que la velocidad lineal máxima funcional es de 0.4 y la mínima de 0.2. En cuanto a la velocidad angular, se establecieron valores de 4.3 como máximo y 0.01 como mínimo.

En el ecosistema ROS (Robot Operating System), utiliza el Sistema Internacional de Unidades, por lo que: En velocidad lineal, siempre es metros por segundo (m/s) y en velocidad angular es (rad/s), que es radianes por segundo.

Rango de funcionamiento lineal

Máximo: 0.4 m/s

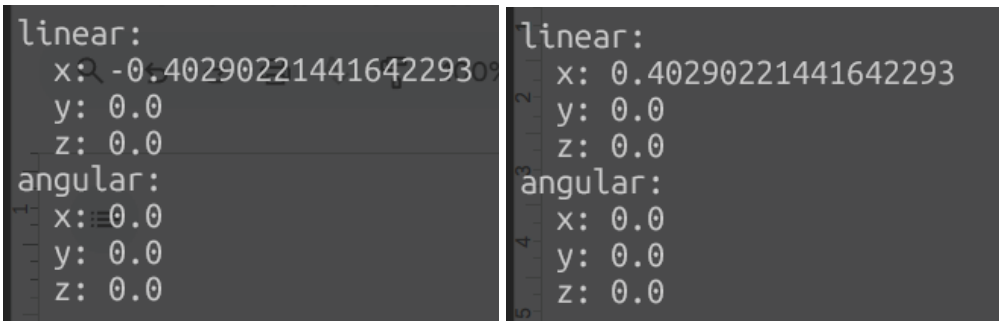

Imagen 4. Máxima velocidad lineal

Mínimo: 0.01 m/s

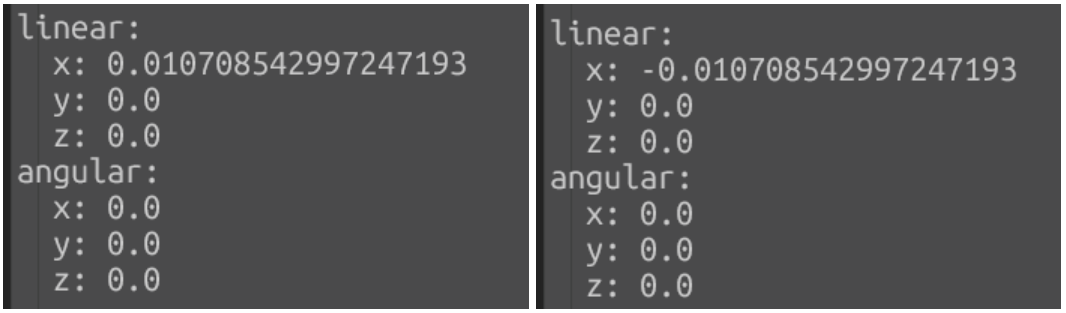

Imagen 5. Mínima velocidad lineal

Rango de funcionamiento angular

Máximo: 4.3 rad/s

```
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -4.347139396894322
```

```
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 4.347139396894322
```

Imagen 6. Máxima velocidad angular

Mínimo: 0.2 rad/s

```
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -0.21960592126606185
```

```
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.21960592126606185
```

Imagen 7. Mínima velocidad angular

Video: https://drive.google.com/drive/folders/1mMvOH9_9uWoQFivMP70XFmw9nAq_hGKa

Conclusiones

En este reto se logró concretar la unión, comunicación y control de forma remota del Puzzlebot mediante el uso ROS2, integrando tanto hardware como software de forma efectiva en un sistema operativo. A través de configurar un hotspot en la Jetson Nano y conectar por SSH nos permitió establecer un vínculo estable que permitió interactuar con el robot en tiempo real sin conexión física.

También se comprendió realmente cómo funciona la arquitectura distribuida de ROS 2 basada en nodos y tópicos, destacando el uso del tópico /cmd_vel para el envío de comandos de velocidad lineal y angular. En este punto la herramienta teleop_twist_keyboard facilitó la teleoperación del robot, permitiendo ver cómo los objetos emitidos por el teclado se convertían en un movimiento físico real.

Por otra parte, las pruebas experimentales permitieron determinar los rangos óptimos de operación del robot considerando aspectos como estabilidad, respuesta y limitaciones físicas del sistema. De este modo se puso en práctica la importancia de la experimentación para validar el comportamiento real.

Finalmente, este proyecto permitió afianzar la comprensión sobre robótica móvil, poniendo bases seguras para desarrollos futuros más avanzados.

Referencias

Puzzlebot: Robotics for All - UoM Innovation Factory. (2024, October 16). University of Manchester Innovation Factory. https://www.uominnovationfactory.com/projects/robotics-for-everyone-puzzlebot/

teleop_twist_keyboard - ROS Wiki. (2024). Ros.org. https://wiki.ros.org/teleop_twist_keyboard